



Experiment – 1.4.2

Student Name: Yatharth Bhaskar

UID: 24IBD70014

Branch: CSE (Data Science)

Section/Group: 24AIT NTTP 1(A)

Semester: 5th

Date of Performance: 08/07/2026

Subject: Full Stack-II

Subject Code: 24CSP-337

1. Aim: To optimize rendering performance and implement testing strategies for interactive UI components.

2. Software Required:

- React.js
- Testing libraries (Jest / React Testing Library)
- Browser DevTools

3. Objective:

- To understand performance bottlenecks in UI systems
- To optimize rendering using memoization techniques
- To reduce unnecessary re-renders
- To implement testing for UI components and logic

4. Theory:

Interactive systems like calendars involve frequent updates and complex UI rendering.

Without optimization, they may suffer from:

- Performance lag
- Excessive re-renders
- Poor user experience

Performance optimization involves:

- Memoization to cache results
- Efficient state updates
- Component-level optimization

Testing ensures:

- Functional correctness
- Stability under different scenarios
- Maintainability of code

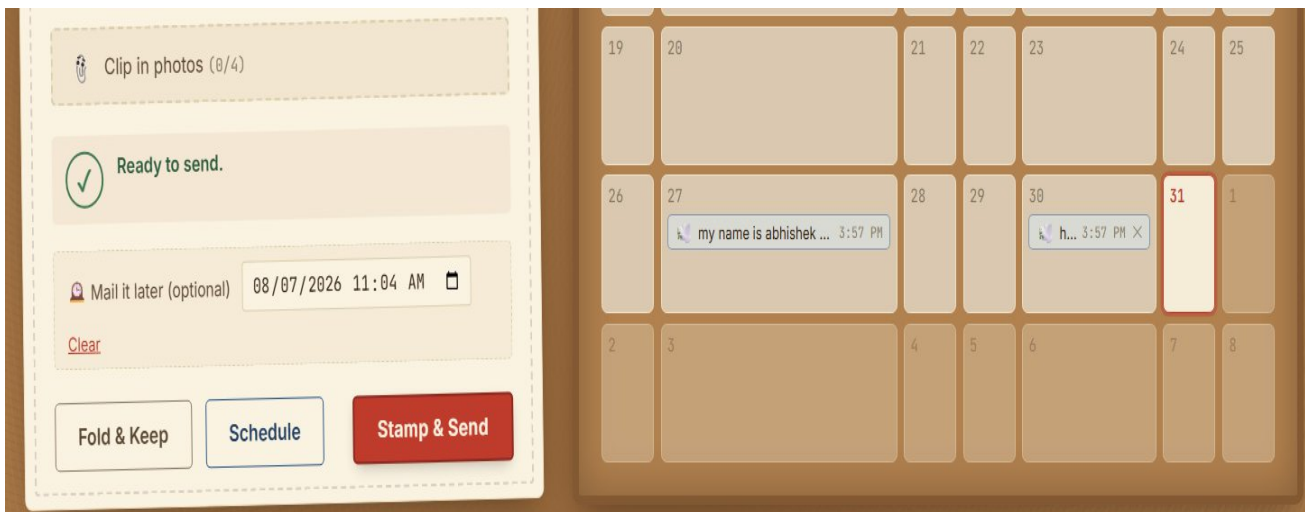
Academic Insight: This experiment integrates Performance Engineering and Software Testing, both critical for building reliable and scalable systems.

5. Procedure/Algorithm:

- Identify performance bottlenecks
- Apply React.memo to components
- Use useMemo for expensive computations
- Use useCallback for stable function references
- Optimize state updates
- Write unit tests for components
- Test user interactions (click, drag, update)

6. Code and Output:

```
client > src > components > Calendar > SchedulingCalendar.jsx > ...
1  import { useCallback, useMemo, useState } from "react";
2  import { useDispatch, useSelector } from "react-redux";
3  import { AnimatePresence } from "framer-motion";
4  import { selectAllPosts, updatePost, setCurrentEditingPost } from
5  import { PLATFORM_ORDER, PLATFORM_CONFIGS } from "../../constants/
6  import { getMonthMatrix, toDateKey, addMonths, isSameDay, formatMo
7  import CalendarPostChip from "../CalendarPostChip";
8  import "../SchedulingCalendar.css";
9
10 const WEEKDAY_LABELS = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri",
11 // Applied when a draft (which has no time of its own yet) gets dr
12 // straight onto a day cell. Rescheduling an already-scheduled pos
13 // its existing time-of-day instead - see handleDrop below.
14 const DEFAULT_SCHEDULE_HOUR = 10;
15
16 const startOfDay = () => {
17   const d = new Date();
18   d.setHours(0, 0, 0, 0);
19   return d;
20 };
21
22 const isPastDay = (date) => {
23   const endOfDay = new Date(date);
24   endOfDay.setHours(23, 59, 59, 999);
25   return endOfDay < new Date();
26 };
27
```



7. Conclusion:

- Optimized calendar rendering
- Reduced unnecessary re-renders
- Stable and testable UI components
- Improved application performance

8. Learning Outcomes:

- Learned to identify and address performance bottlenecks in interactive UI components
- Understood how memoization techniques reduce unnecessary re-renders
- Implemented React.memo, useMemo, and useCallback for component-level optimization
- Gained experience writing unit tests for UI components and user interactions
- Enhanced skills in building performant, stable, and testable React applications